Missing or Misranked? Two Ways a Sampling Solver Loses Right Answers

Satyam Dwivedi[1,2], Nishi Kumari[2], Avinash Pandey[2], Anurag Purushottam[2], Rakhi Verma[2]

[1]Vaikhari AI · [2]Null Hypothesis Enthusiasts*

November 2026

Abstract

A solver that generates many candidate answers and submits a few can lose a right answer in two ways: it is never generated (missing), or it is generated and ranked below the answers submitted (misranked). We ask which dominates, whether that holds beyond our system, why generation stops, what selection can recover and where compute should go, for our ARC-AGI-2 entry: a public Qwen3-4B test-time-training transducer (B0) followed by Qwen3.8-27B on the shortest tasks. It scored 34.31 on the public leaderboard, and 29.31 resubmitted unchanged. On an offline store (the bank) of every candidate either model generated for 101 public evaluation tasks (145 test outputs), the direct merge of the two models submits the right grid for 45 outputs, misranks it for 10 and misses it for 90: a perfect selector would add at most 10, and 85% of the 145 would take 124. Five public pools of B0's recipe give a similar split. At the margin, selection binds: three changes that put more right grids into the pool (a wider B0 search, temperature samples, more 27B samples) added 4, 6 and 6, and none raised the count submitted right (−3, 0 and −2). No selection rule registered in advance (pickers, judges, merge orderings) gained more than 2 outputs on the bank, as much as 800 null replays reached, and the judges lost outputs; on the bank, the one change that added more was a second generator, which lifted B0's 40 to 45. Count both ways before tuning either side.

1. Introduction

Many reasoning systems generate candidate answers and then submit a few: code generators filter samples by tests (Li et al., 2022), mathematics solvers vote over sampled solutions (Wang et al., 2023), and strong recent ARC solvers sample grids from a transducer, a program writer or both, then submit two (Akyürek et al., 2024; Li et al., 2024; Franzen et al., 2025). Such a system can lose a right answer in two ways: it never generates it (missing), or it generates it and ranks it below the answers it submits (misranked). The score does not say which, and the two call for different work. A misranked answer can be recovered offline at no generation cost, because every candidate already exists; a missing one needs more compute, a stronger model or a different method. Published accounts differ on which dominates: Moghe and Chin (2026) find an ARC-AGI-1 pipeline generation-bound, while Bay and Yearick (2026) argue that recognising a right answer, not generating one, is the bottleneck, since coverage keeps rising while selection is capped. On ARC-AGI-2, a public release of NVARC-style candidate pools splits accuracy into coverage times selection efficiency and finds most of the loss from ARC-AGI-1 in coverage [K1].

We study the split for our ARC Prize 2026 entry on ARC-AGI-2, which asks for the output grid of a transformation shown by a few demonstration pairs, with two attempts per test output (Chollet et al., 2025). An offline candidate store, which we call the bank, keeps every candidate that B0, the public NVARC-family notebook (Qwen3-4B with

* Null Hypothesis Enthusiasts is an independent research collective for unusually curious people to pursue important questions with people they might never otherwise meet. It is the team name under which this entry competed in ARC Prize 2026.

test-time training and depth-first search), generated on 101 public evaluation tasks, and the direct samples of Qwen3.8-27B on the shortest of them. A selection rule can then be replayed, that is, rerun on the saved candidates, and each output falls in exactly one of three places: submitted and right; misranked, in the pool but not submitted; missing, never generated (Figure 1). We ask five research questions:

1. **RQ1.** When the solver misses, was the right answer missing or misranked, and how does that vary with task length and difficulty? (Section 3.1)
2. **RQ2.** Does the split hold beyond our pipeline, on independent candidate pools and in other solvers? (Section 3.2)
3. **RQ3.** Why does generation stop: the search rule, the token budget or the model? (Section 4)
4. **RQ4.** How much can selection recover against its ceiling, and can the leaderboard measure it? (Sections 5 and 6)
5. **RQ5.** Where should a fixed 12 hours of compute go? (Section 7)

What was fixed in advance. Before each intervention on the bank ran, selection and generation alike, we wrote down its decision rule (outcomes in Appendix A), and before each entry from Oct 31 was submitted, we committed a predicted leaderboard score; these are the confirmatory parts. For selection the common decision rule is a net gain of at least 3 outputs with at most 1 lost; it sits above the largest gain that 800 null replays (rules that should change nothing) reached, and a rule that met it would then face 19 tasks sealed for one final check, once. The account (RQ1), its cross-checks (RQ2), the explanations of why generation stops (RQ3) and the advice on compute (RQ5) are exploratory: they came from analysis after the runs.

We do not claim the coverage/selection split as new (Section 8). Our contributions are:

1. a per-output account of where a two-generator solver loses its answers, by task length and difficulty, cross-checked on independent public pools (Section 3);
2. the mechanisms that stop generation, a probability floor and a token cap, and what loosening each costs under a fixed budget (Section 4);
3. a test of selection against its ceiling, with every rule registered in advance and a decision rule set above null replays, and a measurement of the leaderboard's own noise (Sections 5 and 6);
4. a recipe, with the candidate store, rules and replay scripts released, for running the same account on other sample-then-select systems (Section 9, Appendix C).

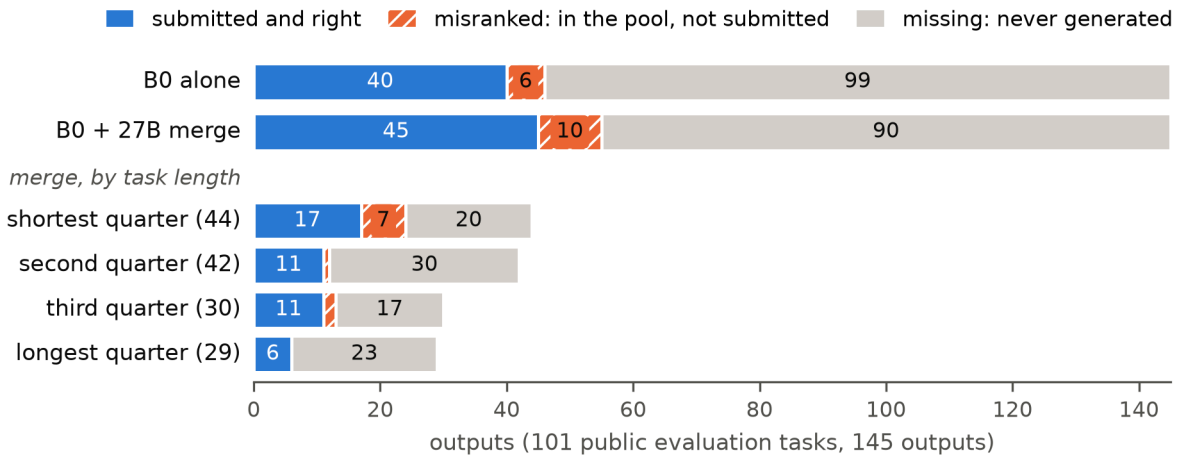


Figure 1: Where each output ends up on the bank (the offline candidate store of Section 3), for B0 alone, the direct B0 + 27B merge, and the merge by task-length quartile; the hatched segment is misranked. Of the merge's 100 misses, 90 were missing (never generated) and 10 misranked.

A companion paper, "The Idea Gap" (Dwivedi et al., 2026), studies agents on ARC-AGI-3. The two papers share no claims.

2. Setting and system

Task and scoring. A task gives a few demonstration pairs and one or more test inputs. A test output scores when either of two submitted grids equals it exactly; the score is outputs right over outputs. Each entry ran on 240 hidden tasks on one machine with four NVIDIA L4 GPUs and a 12-hour limit, and the public leaderboard scores the public half of them.

B0. B0 is a public notebook built on a Qwen3-4B model fine-tuned on grids. For each task it trains a LoRA adapter on augmented copies of the demonstrations: 128 training sequences in the public notebook, 64 in our entry. It decodes each test input under 16 views (8 dihedral transforms times 2 colour permutations) with a depth-first search that keeps a partial grid only while its probability stays above 0.2. It rescores every distinct grid under 8 augmentations and submits the top two by kgmon: the number of views that produced the grid minus its mean augmentation score (a negative log-probability). A task stops when its search runs out of sequences above the floor, or at a 1,200 s cap that includes training and is checked between batches, so a capped task can run past it.

The 27B phase. B0 starts no new task after 9 h 18 min. Qwen3.8-27B then runs (FP8 weights on vLLM with multi-token prediction, 81,920-token context, highest reasoning effort) on tasks in order of task-file length, shortest first, sending requests until 11 h 15 min; the hard stop is 11 h 30 min. In the entry, each reply carries a grid and a Python `transform()`, which runs on the demonstration pairs. One repair round follows a failed check, and one rescue request follows a reply cut at the token cap. The merge puts a program that reproduces every demonstration first; otherwise the 27B's grid replaces B0's attempt 2 when it differs from attempt 1. v7's direct merge, which the bank replays, uses only the second rule.

Entries. Table 1 lists every ARC-AGI-2 entry from B0 on.

Track	Entry	Build	Submission	Public score
ARC-2	B0	public notebook, 128 training sequences	57387075	30.56
ARC-2	H1	B0 with 64	57518886	28.89
ARC-2	v1	H1, then the 27B direct, one sample	57541839	31.39
ARC-2	v2	v1, second samples on the shortest quarter	57582551	31.81
ARC-2	v6nb	B0 at a quarter of the training and half the views	57610305	28.61
ARC-2	**v7-vr** (final)	v2's B0; 27B with verify and refine	57646090; 57785824	34.31; 29.31 (identical)
ARC-2	v7	v2's B0; 27B direct, deadline caps	57682082	30.97
ARC-2	v7-vr2	v7-vr, written grids above unverified programs	57716508; 57752087	32.64; 32.22 (identical)
ARC-2	S1	v7-vr2, 128 training sequences, B0 phase 5,630 s longer	57837368	31.81
ARC-2	**zbeam** (final)	v7-vr2, attempt-2 picker	57853811	32.64
ARC-2	amax	v7-vr2, another attempt-2 picker	57887325	29.31 (after close)

Table 1: ARC-AGI-2 entries in submission order; "final" marks the two final selections. "Identical" marks a byte-identical resubmission of the same notebook version. No difference in this table is a measured gain (Section 6).

3. Where the misses are

3.1 The account (RQ1)

RQ1 asks where each miss sits. To build the bank, B0 ran at the entry's setting (64 training sequences, 16 views) on the 120 public evaluation tasks. Nineteen tasks were sealed for one final check; the other 101 tasks, with 145 test outputs, form the bank. For every output it keeps each candidate grid with its beam score, its 8 augmentation scores and its kgmon rank, and B0's two attempts. The 27B adds direct samples on the band, the 30 shortest of the 120 tasks by task-file length (44 outputs): 55 samples on 29 of the 30 tasks, at a 49,152-token cap. Ground truth is the competition's copy of the evaluation solutions. A selection rule is then a function from saved candidates to two attempts, replayed on CPU in seconds, with no run noise.

Three places. For a rule with candidates *C* and attempts *a1*, *a2*, an output is *right* if its true grid is *a1* or *a2*; *misranked* (in the pool) if the true grid is in *C* but not submitted; *missing* (never generated) otherwise. For B0 alone, *C* is B0's candidates. For the merge, *C* adds the 27B's grids.

Track	Group (tasks, outputs)	B0 alone: right / misranked / missing	Merge: right / misranked / missing
ARC-2	all (101, 145)	40 / 6 / 99	45 / 10 / 90
ARC-2	shortest quarter (30, 44)	12 / 3 / 29	17 / 7 / 20
ARC-2	second quarter (30, 42)	11 / 1 / 30	11 / 1 / 30
ARC-2	third quarter (22, 30)	11 / 2 / 17	11 / 2 / 17
ARC-2	longest quarter (19, 29)	6 / 0 / 23	6 / 0 / 23
ARC-2	player difficulty 1 (4, 5)	4 / 0 / 1	5 / 0 / 0
ARC-2	player difficulty 2 (18, 22)	13 / 1 / 8	13 / 1 / 8
ARC-2	player difficulty 3 (45, 63)	16 / 3 / 44	17 / 7 / 39
ARC-2	player difficulty 4 (25, 38)	6 / 1 / 31	7 / 1 / 30
ARC-2	player difficulty 5 (9, 17)	1 / 1 / 15	3 / 1 / 13
ARC-2	all, five public pools of B0's recipe [K1] (101, 145)	24 to 37 / 6 to 9 / 102 to 112	–

Table 2: The account per output on the bank. "Merge" is v7's direct merge (B0's attempts, attempt 2 replaced by the first 27B grid that differs from attempt 1). Quartiles are by task-file length over the 120 public tasks. The last row is the range over the five public pools, scored with their own scorer.

The 27B adds grids only where it runs. In the shortest quarter the merged pool holds 24 right grids against B0's 15, and the merge submits 17 against B0's 12. The other three quarters are identical for B0 and the merge, since the 27B has no samples there. Seven of the merge's 10 misranked outputs sit there.

Harder tasks are almost all never generated. Members of the team played the evaluation tasks individually, by hand, and each task carries one difficulty label from 1 (easiest) to 5 (hardest). By output (Table 2, Figure 2), the never-generated share of the merge's misses is 8 of 9 at difficulty 2, 39 of 46 at 3, 30 of 31 at 4 and 13 of 14 at 5.

RQ1. Answer: missing. 90 of the merge's 100 misses and 99 of B0's 105 were never generated, so perfect selection would reach 55 of 145, against 124 for 85%; 7 of the 10 misranked outputs sit in the shortest quarter, and at difficulty 4 and 5, 43 of 45 misses were missing.

3.2 Beyond our system (RQ2)

RQ2 asks whether the split comes from our pipeline or scorer, and whether other solvers behave alike. Independent pools of the same recipe rule out the pipeline and the scorer. The public "Out of Reach" release [K1] holds five pools of B0's 4B recipe, four rebuilt to the deadline and one from a public full-clock notebook. On the bank's 145 outputs they leave 102 to 112 missing and 6 to 9 misranked (Table 2), against B0's 99 and 6, and their scorer on B0's pool returns our 40; 78 outputs are missing from all seven pools (theirs, B0's, the 27B's). Pools of one recipe are not interchangeable, though: the rebuilt submitted configuration covers 37 outputs against B0's 46 (McNemar p 0.049), partly because it drops grids decoded at the deadline.

Two earlier solvers point the same way (Appendix B). Small transducers trained from scratch (a GRU, Mamba-2 and a Mamba-2/attention hybrid against a Transformer, 0.7M to 21M parameters) solved none of 53 held-out test outputs from training tasks new in ARC-AGI-2, except one hybrid seed with 1: a new generator architecture created almost no coverage there. A program-search solver tried eight small learned priors that reorder its search; none solved any of 85 held-out outputs from training tasks new in ARC-AGI-2, and at 10 s per task three of them scored 0 on our two 40-task evaluation splits. Its own oracle check, on the one task it covers, shows why ordering could not help: given the correct family and perception, the generator yields 1,191 programs and no projection proposal.

RQ2. Answer: for B0's recipe, yes: independent pools, scored with their own scorer, give a similar split. The two earlier solvers, scored on other outputs, are consistent with it: neither a new generator architecture nor a learned search order solved more than 1 output from tasks new in ARC-AGI-2.

4. Why the right grid is never generated (RQ3)

RQ3 asks why generation stops: the search rule, the token budget or the model. The mechanisms below come from analysis after the runs; the interventions that test them had decision rules fixed in advance (Appendix A).

4.1 B0 stops on probability, not on time

An audit of B0 unchanged (128 training sequences) on 40 evaluation tasks (65 test outputs) set the pattern. It submitted the right grid for 12 outputs, and its first attempt alone was also right on 12, so the second attempt never added a solve. The right grid was anywhere in its pool for 15 and never generated for 50. Every task stopped because the search ran out of sequences above the 0.2 floor, not on a time cap (median 601 s, longest 1,379 s per task). On the bank, at 64 training sequences, 94 of 101 tasks ended the same way and 7 at the 1,200 s cap (median 469 s).

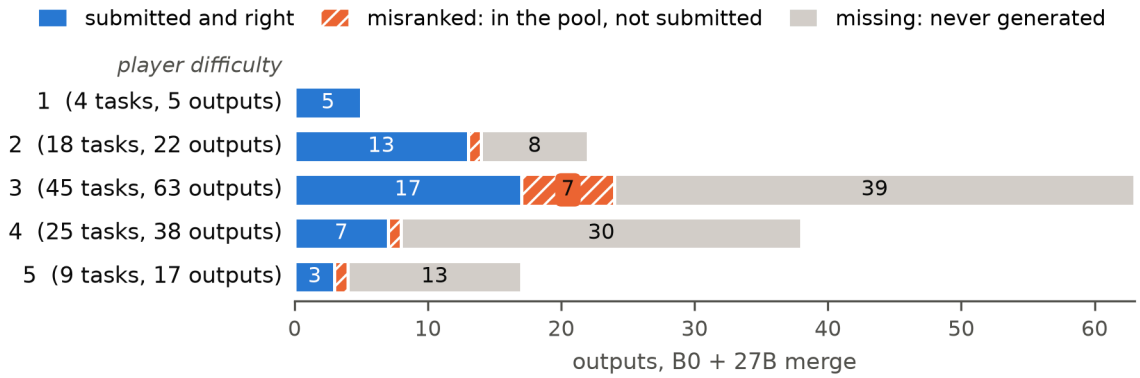


Figure 2: The merge's account by the team's difficulty label (1 easiest, 5 hardest); the hatched segment is misranked. Seven of the 10 misranked outputs are at difficulty 3; at 4 and 5, 43 of 45 misses were never generated.

A lower floor finds more grids but does not fit the limit. At a floor of 0.05 on the audit's tasks, the pool grew from 15 to 19 right grids and the submission from 12 to 14, but each task took 1.66 times as long, 20 of 40 tasks hit the cap, and the projected 240-task run grew from 10.6 h to 17.6 h. An unchanged repeat of the audit also reached 19 and 16, gaining the same two outputs, so even this gain is within B0's run-to-run variation.

We then searched B0 on the bank once with a superset setting (32 views, floor 0.1, 32 rescoring augmentations) and replayed narrower settings from its logs under the bank's caps, paired against the bank's setting replayed from the same run (Figure 3). The pool grew from 48 right grids to 50 at floor 0.1 (1.37 times the time), 49 with 32 views (1.43 times) and 52 with both (1.76 times). B0's own two attempts did not follow: 42 at the bank's setting, 39, 42 and 39. With the extra B0 time charged against the 12-hour run, no 27B sample would survive and the projected merge would fall from 48 to 36, 38 and 31. No setting reached a net gain in the projected merge on the half of the tasks used to choose, so the bank's decoding stayed.

Temperature sampling gave the same shape. After B0's search, two rounds of samples at temperature 1.0 per view, on the same adapters, found right grids for 6 outputs the search never generated: the pool grew from 45 to 51 on 142 outputs. B0's two attempts stayed at 39 (one gained, one lost), at 1.99 times B0's time per task; one round gave a pool of 47 at 1.50 times. Retraining did not help: 128 training sequences instead of 64 gave 36 against 40 on the bank at 1.17 times the time, fixing colour 0 in every colour permutation gave 39, and adapting B0's model on 80 other evaluation tasks left the audit's tasks at 15 of 65. Halving training from 128 to 64 sequences had kept 15 of 65 at 18% less time per task, which is why our entry runs B0 at 64.

4.2 The 27B runs out of tokens

The 27B's first direct sample is right on 17 of 44 outputs from the shortest quarter of tasks by length and on 2 of 46 beyond it. Longer tasks need longer reasoning, and the reasoning is cut. On 40 development tasks, 70 of its 80 samples ran out of their thinking budget and needed a rescue request, at 97k generated tokens per task. On the 50 bank tasks beyond the band, 92 of 100 samples at the 49k cap were cut and rescued.

A longer cap generates more right grids where tokens are the binding limit (Table 3). On the next 20 tasks by length after the band (26 outputs), one sample at a 98,304-token cap was right on 8; two 49k samples on the same outputs were right on 4 as a union. Every right answer came from a reply that ended its own reasoning: such answers were right on 8 of 10 outputs, answers cut at the cap and rescued on 0 of 9, and outputs with no grid on 0 of 7. The cost was 79.6k generated tokens per task at 221.5 tokens per second with 12 requests in flight, 7.2 ks for the 20 tasks.

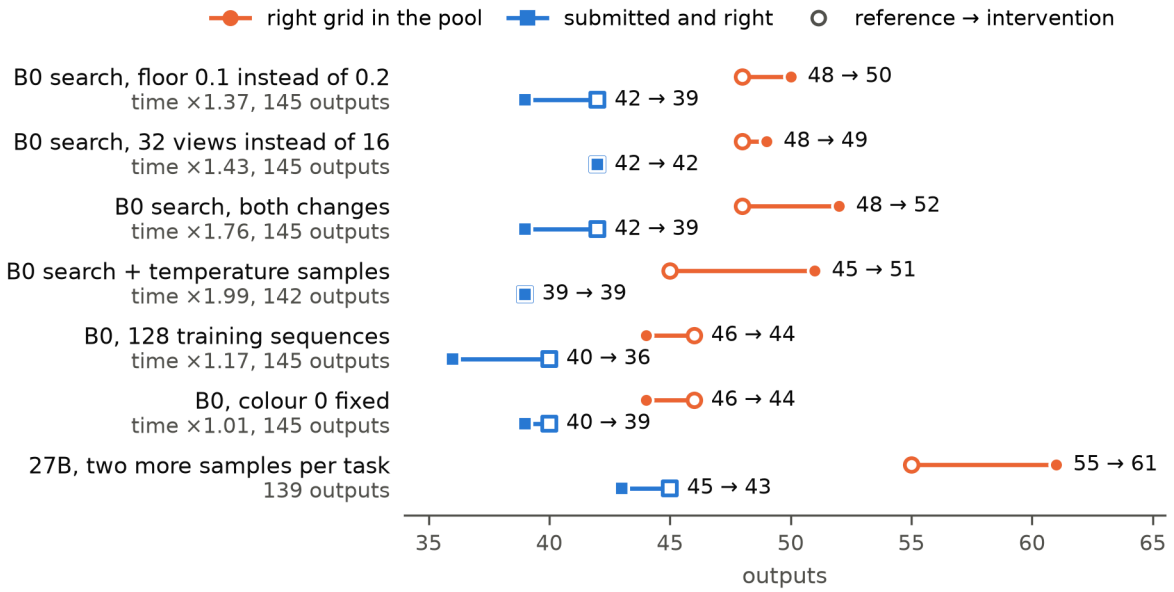


Figure 3: Generation interventions: the right grid in the pool (circles) against submitted and right (squares), hollow for the reference and filled for the intervention, on the same outputs. The references are the bank's setting replayed from the same run (the B0 search changes), B0's search alone (temperature samples) and the bank (the rest). Wider search and temperature samples put more right grids into B0's pool, and its two attempts did not take them; more 27B samples grew the merged pool and lowered the merge.

Track	Run	Outputs	Finish state	Right
ARC-2	98,304-token cap, next 20 tasks after the band	10	ended its own reasoning	8
ARC-2	98,304-token cap, next 20 tasks after the band	9	cut at the cap, then rescued	0
ARC-2	98,304-token cap, next 20 tasks after the band	7	no grid	0
ARC-2	two 49,152-token samples, same 26 outputs	26	union of both samples	4

Table 3: The 27B's yield by finish state, one sample at temperature 1.0 per output (two in the last row). The 18 misses at the 98,304-token cap: 9 cut, 7 with no grid, 2 wrong grids.

Rescues and forced answers did not replace the missing reasoning: the full saved reasoning instead of its last 16,000 characters gave the same 14 of 25, worked programs in the rescue lost 3 net over 196 paired draws, and a forced answer at a 40,960-token budget scored 9 of 25 against 14 and 10.

4.3 The two models fail on different tasks

On 40 development tasks, the 27B's two direct samples were right on 9 outputs, and 7 of them were outputs B0 missed in both of its runs; the union reached 23 against 16. The tasks it solved sat at ranks 9 to 31 of 40 by B0's own

confidence, so routing B0's least confident tasks to the 27B does not reach them. Routing by length does. On 20 unseen tasks fixed by length before the run (25 outputs), B0 with 64 training sequences was right on 9, the 27B on 14 and the merge on 16 (9 gained, 2 lost). The 27B as a program-writing agent that checks its programs on the demonstrations solved 3 of 65 outputs, all of them also B0's, and used up its token budget on 38 of 40 tasks. Other models in its place generated less on the routing tasks: Muse Glimmer 30B 8 of 25 (variants 5 to 11) against 9 to 14 for the 27B, and Qwen3.8-Flash-Next 4 of 25 at 99 tokens per second against the 27B's 334.

RQ3. Answer: the search rule for B0 (its probability floor, on 94 of 101 tasks) and the token budget for the 27B (92 of 100 samples beyond the band cut). Loosening either limit added right grids in single runs, at a cost in time or tokens; the 27B mattered because it solved outputs B0 never generated.

5. Selection against its ceiling (RQ4)

RQ4 asks first how much selection can recover. The account bounds it: at most 6 outputs for B0 alone (46 in its pool against 40 submitted) and 10 for the merge (55 against 45). Among the 46 outputs whose right grid is in B0's pool, kgmon ranks it first on 29, second on 11, third on 4, fourth on 1 and lower on 1; on the bank, B0's attempt 1 alone is right on 29 outputs and both attempts on 40. Every rule below was registered with its own decision rule before its replay; for the bank's selection rules the common one is a net gain of 3 outputs or more with at most 1 lost, inside a family of rules registered together (the first rules, on Oct 27, used an earlier one: net +4 and a sign test). Figure 4 places each bank rule against the base and the ceiling.

Pickers from B0's own signals. Three rules for B0's second attempt used only scores already in the pool: zbeam (z-scored view count, mean augmentation score and best beam score), amax (view count minus the worst augmentation score) and borda (kgmon and probmul3 ranks). On six earlier development pools (390 outputs) they scored 90, 89 and 87 against kgmon's 86. On the bank, zbeam reached 41 alone (2 gained, 1 lost) and 47 merged (2 gained, 0 lost); amax changed attempt 2 on 29 outputs and was right on none of them. Neither met the common decision rule; amax met its own entry rule (no net loss, at least one attempt changed) and was submitted once.

External judges. Glimmer 30B's log-likelihood of each candidate, added to B0's score on the 72 outputs where B0 was least sure, gave 35 (1 gained, 6 lost). A 2B decision model choosing zero-shot among B0's candidates gave 34 to 36 under three of its rules and 25 under the fourth; on the 42 outputs where the right grid was among two or more candidates, its top pick was right on 15 against B0's 25. Asked to check or correct B0's top two candidates, the 27B scored 10 of 25 against 14 for its own answers (12 against 16 as the merge's attempt 2): it found slightly more right candidates (6 of 9 against 5) but lost its own solves when all were wrong (3 against 7), so it served better as a generator than as a checker.

Merge orderings. Putting the 27B's grid first, or first when its two samples agree, leaves the merge at 45 on the bank; only attempt-1 hits move (33 and 32 against 29), which pass@2 does not reward.

Program checks add a little. Programs that reproduce the demonstrations are the only selection signal that scored positive, by 1 to 2 outputs and within noise. On the 20 routing tasks, verified programs covered 6 outputs and 5 were right; preferring them over the written grid gave 15 of 25 against 14. The entry as first built ranked unverified program outputs above written grids and replayed at 13 against the direct merge's 16; v7-vr2 ranks written grids first and restored 16. On the 30 band tasks (44 outputs), two replays of the entry's 27B phase gave 17 and 19 for v7-vr and 18 and 19 for v7-vr2, against 17 for the direct merge and 12 for B0; verified answers were right on 25 of 33. That replay failed its validity check (8 of 132 records timed out with two replicates on one server), so it is reported, not used for a decision.

More samples without checks. Two more 27B samples per task, on the band and beyond it, raised the merged pool from 55 to 61 but lowered the merge from 45 to 43: beyond the band, unverified wrong 27B grids replaced 4 right second attempts of B0, against 2 gained. (Scored on 139 outputs: three tasks where one sample answered a grid larger than 30 × 30 are left out, and none of their outputs is right in any pool.)

Chance. In 800 replays of four level-preserving null rules (score jitter at three levels, and the order of the 27B's samples), no seed reached net +3; the largest was +2. The best rule we found, zbeam in the merge at +2, is inside that range.

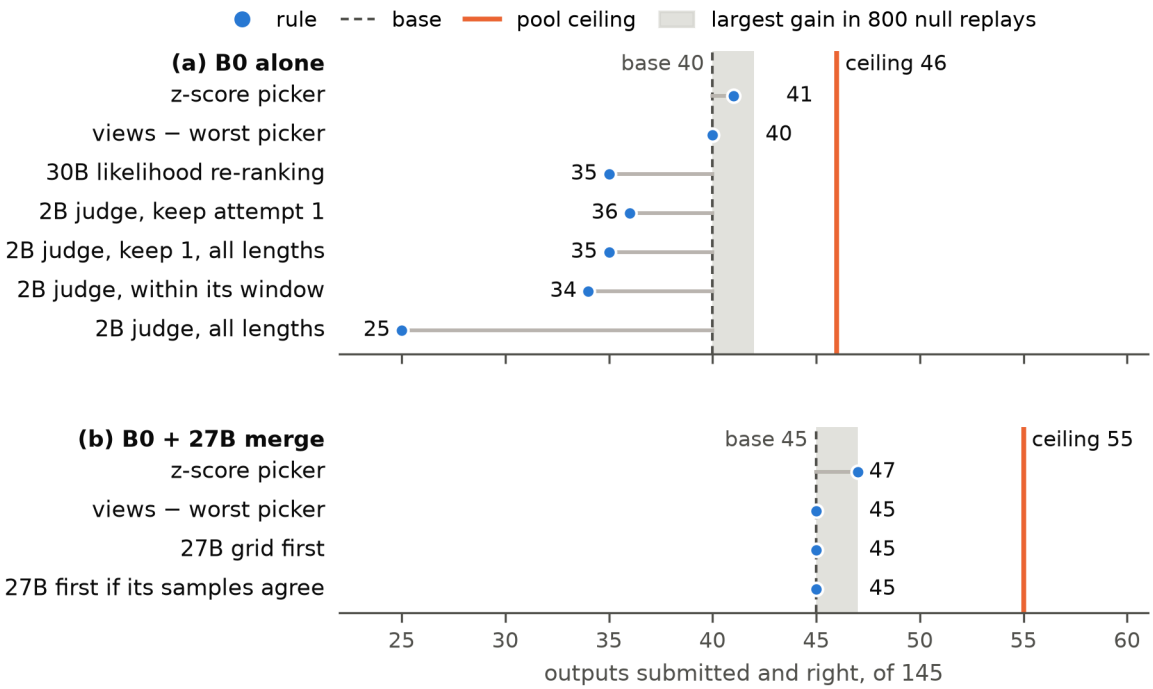


Figure 4: Selection rules on the bank against the base (dashed) and the pool ceiling (solid), in outputs submitted and right of 145, for (a) B0 alone and (b) the merge; the grey band is the largest net gain in 800 null replays. The z-score and views − worst pickers are the zbeam and amax rules of Section 5; the 2B judge's four rules either keep B0's attempt 1 or not, and score either the requests that fit its 4,096-token window or all lengths.

RQ4, recovery. Answer: little: the ceiling is 10 outputs, no registered rule met the decision rule, and the best gained 2, as much as null replays reach.

6. Measuring without the leaderboard (RQ4)

The second half of RQ4 is whether the leaderboard can see gains of this size. It is noisy: v7-vr, resubmitted byte for byte (same kernel version, notebook and inputs), scored 34.31 and 29.31; v7-vr2 scored 32.64 and 32.22; four byte-identical copies of B0 on the leaderboard, ours and three published by others, scored 28.06 to 32.22. A difference between two hybrid scores has an SD of about 3.2 points (2.6 for B0-only copies). Before any run noise, task sampling alone gives one public score an SD of 3.70 points (4,000 bootstrap resamples of 120 bank tasks, the size of the public half), and two builds must differ by about 4.4 points to be told apart (1.96 times 2.25, the SD of a paired difference). Figure 5 shows every entry against that spread. After the repeat, verify and refine scores about +1 over the direct merge, not the +3.34 of the first pair.

B0 is not reproducible from draw to draw. On 616 views decoded by the audit and by the floor-0.05 run, whose grids above 0.2 should match exactly, the set of grids above 0.2 differed in 54%. Rerunning the bank's own setting gave 39 against the bank's 39 on the same 142 outputs (2 gained, 2 lost), with the same candidate set on only 38 of 145 outputs and the same two attempts on 80; a second rerun gave 42 against 40 on all 145 (3 gained, 1 lost). A change that needs a new GPU run, such as more training, is therefore compared through draw noise even on the bank. A selection rule replayed on saved pools is not.

The pre-registered protocol. A rule is registered before its replay. The decision rule (net +3 or more, at most 1 lost) sits above the null replays, where net +2 occurs in under 5% of seeds and net +3 in under 1%; a rule that passed

would then need the same sign on the 19 sealed tasks, once. No bank rule passed. The sealed tasks were scored once, for zbeam as a report: net −1 on the 19 (0 gained, 1 lost) and net 0 on all 172 public outputs; no untouched sealed check remains.

Calibration. Four entries have both a bank replay and a leaderboard score. They fit leaderboard ≈ 8.84 + 0.727 × bank score (points), residual SD 0.42, leave-one-out error 0.52 on three of them; but the four replays take only two distinct bank values, so the fit is close to a line through two points.

Predictions committed before scoring. From Oct 31, every entry carried a prediction committed before submission. For S1 (128 training sequences, with B0's phase 5,630 s longer), the offline time model projected about 26.9; it scored 31.81, so either the projected time cut did not hold on the hidden tasks or the bank's B0 loss did not carry, and one score cannot separate the two. For zbeam, about 33.8; it scored 32.64. For amax, which the bank scored as no change, about 32.4; it scored 29.31, 0.3 below the no-change band, more likely run noise than the picker, since its attempt 2 is right on none of the 29 bank outputs it changes. Development replays had overstated selection gains: zbeam's +4 of 390 outputs became +1 of 145 on the bank, and amax's +3 became 0.

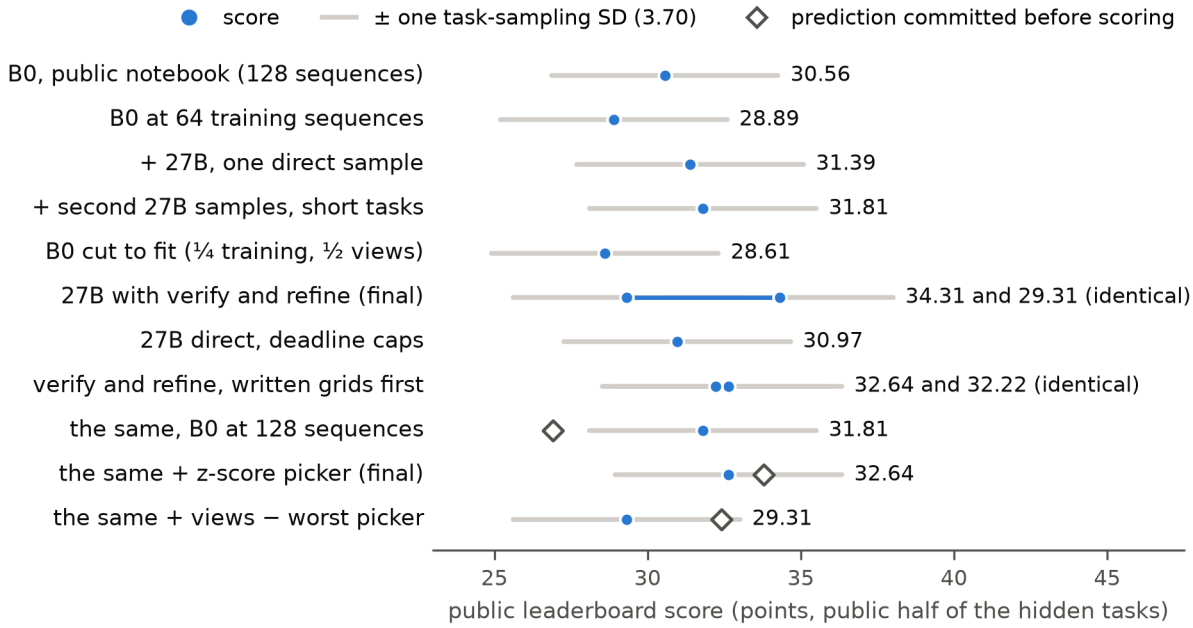


Figure 5: Public scores of the ARC-AGI-2 entries in submission order, with one task-sampling SD and the predictions committed before three of them were scored; a line joins the two scores of a build resubmitted unchanged, and the last row was scored after the competition closed. Rows follow Table 1: B0, H1, v1, v2, v6nb, v7-vr, v7, v7-vr2, S1, zbeam and amax.

RQ4, measurement. Answer: the leaderboard cannot measure it. Replaying saved candidates removes run noise and sets a decision bar above null replays; no rule cleared it, and the one sealed check did not confirm the best rule.

7. Where the 12 hours went (RQ5)

RQ5 asks where a fixed 12-hour budget should go. Our entry gave B0 the machine until 9 h 18 min and the 27B the rest. On development timing, B0 at the public notebook's 128 training sequences projected to 10.6 h for the 240 hidden tasks, and 8.84 h at 64. The bank's throughput model, with start-up and overhead, predicts 9.96 h at 64 against the 9.30 h phase, enough to cut about 10 of 240 tasks. The entry that cut B0 further to make room (v6nb) scored 28.61, 3.20 below v2 on single scores.

The 27B produced about 334 generated tokens per second with 20 requests in flight, at about 45k tokens per task, which after B0 reaches about the shortest 52 tasks by 11.5 h in that run's projection. Its yield falls with length (Section 4.2). More 27B samples raised the pool more inside the band (24 to 28 right grids) than beyond it (31 to 33,

where 10 of the 27B's 12 right grids duplicate B0's). Per-token gains need not survive a fixed budget either: in our recurrent-model study (Appendix B), a Mamba-2/attention hybrid beat a Transformer of its size at equal training tokens (38 and 47 exact held-out outputs against 24), but one Transformer seed given the same training time solved 55, against 38 (p 0.008) and 47 (p 0.18), because the hybrid's scan ran slower per token in our implementation.

Outside evidence. Competitors' reports after the deadline point the same way; their numbers are author-reported, not reproduced here. No top-10 final notebook was public on Nov 3, when the public notebooks we read from code (not run, possibly not the scored versions) gave 11 to 12 hours on the same 4×L4 machine to one strong model; by Nov 5 the solution reports of public-leaderboard ranks 2, 4 and 9 and rank 4's notebook were public. Rank 2 reports that a second independent solver added 6 to 7 points, at 200k to 300k tokens per conversation and about 46M tokens per 12-hour run [K2]; rank 4 makes two independent tries of up to 200k tokens [K3]; rank 9 runs two independent passes that vote and reports Flash-Next strong as long as it generates enough tokens (256k) [K4]. Those budgets are four to six times our 27B's 49k cap. The same Qwen3.8-27B scores 26.81 with two 56k attempts and 44.03 after fine-tuning on its own verified traces [K6]; ours ran untuned, after B0's 9 h 18 min. Ranks 2 and 4 serve Flash-Next at 3 and 2 bits, rank 2 with pruned experts [K2, K3]; our own attempt to serve Flash-Next reached 99 tokens per second (Section 4.3), so such serving work is part of what the account asks for.

RQ5. Answer (exploratory): toward generation by the stronger model, with a check that can lift its right grids into the submitted attempts. Every generation intervention that worked bought its grids with time or tokens, and competitors report the same direction, but we did not test a reallocation: the entry that cut B0 scored lower, the doubled cap was never submitted, and checks gained 1 to 2 outputs, within noise.

8. Related work

ARC. Chollet (2019) proposed ARC as a measure of skill-acquisition efficiency; ARC-AGI-2 raises its difficulty with tasks calibrated on human testers (Chollet et al., 2025). Test-time training of a transducer on each task's demonstrations (Akyürek et al., 2024), with augmented views and depth-first sampling scored across views (Franzen et al., 2025), underlies B0, a public notebook in the NVARC family (Sorokin and Puget, 2025). Li et al. (2024) found that induction (writing programs) and transduction (writing grids) solve different ARC tasks. In our system two transducers, B0 and the 27B answering directly, differed in the same way, while the 27B writing programs added nothing B0 lacked (Section 4.3). Greenblatt (2024) and Pourcel et al. (2025) sample and refine programs checked on the demonstrations.

Repeated sampling and selection. Pass@k separates what a model can generate from what is submitted (Kulal et al., 2019; Chen et al., 2021). AlphaCode filters and clusters samples by test behaviour (Li et al., 2022), CodeT ranks code by generated tests (Chen et al., 2023), self-consistency votes over reasoning paths (Wang et al., 2023), and trained verifiers rank solutions (Cobbe et al., 2021). Brown et al. (2024) show coverage growing with samples while common selection methods plateau; Stroebl et al. (2024) show imperfect verifiers capping the gain from resampling; Snell et al. (2024) study how to spend test-time compute. On ARC-AGI-2, the "Out of Reach" release [K1] splits accuracy into coverage times selection efficiency over five NVARC-style pools, finds efficiency flat and most of the loss from ARC-AGI-1 in coverage, ties coverage to empty pools, and derives a scheduling identity for the 12-hour rerun: our account for B0 alone, published first, and on our outputs its pools reproduce ours (Section 3.2). Its efficiency figures come from a mean negative log-likelihood ranker, not the pools' own selector, which selects 1 to 4 more; and empty pools explain differences between configurations, not the level, since 78 of B0's 99 missing outputs have non-empty pools. Moghe and Chin (2026) rest their account on an unbiased pass@k analysis on ARC-AGI-1, and Bay and Yearick (2026) theirs on coverage that keeps rising with samples while selection is capped.

Evaluation noise. Benchmark scores vary with data sampling and run randomness (Bouthillier et al., 2021; Miller, 2024). We measure both for one competition leaderboard and use an offline replay with a calibrated decision rule in its place.

9. Discussion

Our counts fit both published accounts: most misses are missing, as on ARC-AGI-1 (Moghe and Chin, 2026), while the right grids that more generation added were mostly misranked, as a selection cap predicts (Bay and Yearick, 2026). Which side binds may depend on coverage. In our shortest quarter, where both models generate, the pool holds the right grid for 24 of 44 outputs and 7 were misranked; elsewhere 31 of 101 and 3 (Fisher p 0.08, two-sided). The excess comes with the merge's one slot for the 27B, so coverage there is confounded with the second generator; we leave this as a hypothesis for counts from other systems.

The account needs no ARC-specific machinery; code generation with unit tests and mathematics with answer checkers have the same shape (Chen et al., 2021; Li et al., 2022; Brown et al., 2024). For such systems:

1. **Save every candidate once** with the selector's scores, and seal a share of the tasks for one final check.
2. **Count both ways per output** before tuning either side, by the axes that drive cost (length, difficulty).
3. **Register interventions with decision rules** set above null replays, and count the rules screened.
4. **Measure generation interventions twice,** in the pool and in the submission, charging their time to the fixed budget.
5. **Add a second generator or a check, not a judge.** In our data a second model with its own attempt added 5 outputs, more samples without a check lost 2, the demonstration check helped slightly, and scores borrowed from other models hurt. Competitors report the same: a refiner that checks candidates on the training pairs added 4.6 points (95% CI 1.7 to 8.3) [K2], rank 4 answers only from a function that passes every training pair [K3], and an entry gating submissions on verification reports about 87% of them right [K5], against our 25 of 33 (Section 5; report only).
6. **Treat the leaderboard as a noisy check:** resubmit one build unchanged, and commit a prediction before each submission.

10. Conclusion

We asked where a sample-then-select solver loses its right answers. Our hybrid loses them before selection: 90 of the merge's 100 misses were never generated (RQ1). Independent pools of the same recipe give a similar split, so neither our pipeline nor our scorer explains it (RQ2). Generation stops at a search rule and a token budget, not at the clock per task (RQ3). Selection had at most 10 outputs to recover, no rule registered in advance met the common decision rule, and the leaderboard could not have shown a gain of that size (RQ4). Our data point the 12 hours toward generation by the stronger model with a check, a direction we did not test (RQ5). The candidate store, the rules as registered and the replay scripts are released (Appendix C), so the same count can be run on other systems.

Limitations

- Leaderboard scores are on the public half of the hidden tasks, mostly one score each.
- The bank's tasks are public. They may be in the 27B's training data, and B0's model was trained on synthetic data built from descriptions of them, so the bank ranks rules and measures differences rather than the hidden level.
- The bank replays v7's direct merge, not the entry's checked merge. The program-check results come from separate replays on 25 and 44 outputs, one of which failed its validity check.
- Most 27B results are one sample at temperature 1.0; one such sample on 25 outputs moves by about 4 outputs between runs.
- The 98k cap and the extra samples were measured on subsets and never submitted.
- The difficulty labels are the team's own, one per task; we use them only as a grouping.
- The account, the mechanisms and the compute advice (RQ1, RQ2, RQ3, RQ5) are exploratory; only the decision rules and the committed predictions were fixed in advance, and the null replays calibrate the selection rule rather than test a hypothesis.

- "Never generated" is relative to this system's pool. Another generator changes the split; the counting procedure, not the 90, is what carries over.

Ethical considerations

Difficulty labels. The difficulty labels of Section 3 come from the team's own play of the public evaluation tasks: each member played the tasks individually, by hand, and each task carries one label. No personal data are involved. We use the labels only to group outputs (Table 2, Figure 2), and no other result depends on them.

Data and licences. The ARC-AGI-2 tasks and B0's model are released under Apache-2.0; the other models were used under their published terms, and no weights are redistributed. Our code is released under MIT-0, and the candidate pools, samples, rules and result files under CC BY 4.0, with attribution to ARC Prize for the tasks (Appendix C). Competitors' numbers are quoted from their public posts and marked as their own reports.

Compute. Each of the 13 submissions in Table 1 ran for up to 12 hours on one machine with four NVIDIA L4 GPUs; building the bank and testing interventions used further runs on the same hardware, and every replay and analysis ran on CPU.

References

Akyürek, E., Damani, M., Qiu, L., Guo, H., Kim, Y., and Andreas, J. (2024). The surprising effectiveness of test-time training for abstract reasoning. arXiv:2411.07279.

Bay, Y. Y., and Yearick, K. A. (2026). When more sampling hurts: The modal ceiling and correlation ceiling of test-time scaling. arXiv:2606.28661.

Bouthillier, X., Delaunay, P., Bronzi, M., Trofimov, A., Nichyporuk, B., Szeto, J., Sepah, N., Raff, E., Madan, K., Voleti, V., Kahou, S. E., Michalski, V., Serdyuk, D., Arbel, T., Pal, C., Varoquaux, G., and Vincent, P. (2021). Accounting for variance in machine learning benchmarks. MLSys.

Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A. (2024). Large language monkeys: Scaling inference compute with repeated sampling. arXiv:2407.21787.

Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., and Chen, W. (2023). CodeT: Code generation with generated tests. ICLR.

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. (2021). Evaluating large language models trained on code. arXiv:2107.03374.

Chollet, F. (2019). On the measure of intelligence. arXiv:1911.01547.

Chollet, F., Knoop, M., Kamradt, G., Landers, B., and Pinkard, H. (2025). ARC-AGI-2: A new challenge for frontier AI reasoning systems. arXiv:2505.11831.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. (2021). Training verifiers to solve math word problems. arXiv:2110.14168.

Dwivedi, S., Kumari, N., Mishra, D., Pandey, A., Purushottam, A., Rajput, P., Tripathi, P., and Verma, R. (2026). The Idea Gap: Why open LLM agents fail new environments. ARC Prize 2026 Paper Track, Kaggle.

Franzen, D., Disselhoff, J., and Hartmann, D. (2025). Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. ICML. arXiv:2505.07859.

Greenblatt, R. (2024). Getting 50% (SoTA) on ARC-AGI with GPT-4o. Blog post.

Kulal, S., Pasupat, P., Chandra, K., Lee, M., Padon, O., Aiken, A., and Liang, P. (2019). SPoC: Search-based pseudocode to code. NeurIPS.

Li, W.-D., Hu, K., Larsen, C., Wu, Y., Alford, S., Woo, C., Dunn, S. M., Tang, H., Naim, M., Nguyen, D., Zheng, W.-L., Tavares, Z., Pu, Y., and Ellis, K. (2024). Combining induction and transduction for abstract reasoning. arXiv:2411.02272.

Li, Y., Choi, D., Chung, J., Kushman, N., et al. (2022). Competition-level code generation with AlphaCode. Science 378(6624).

Miller, E. (2024). Adding error bars to evals: A statistical approach to language model evaluations. arXiv:2411.00640.

Moghe, K., and Chin, P. (2026). Cost-effective agent harnesses for abstract reasoning and generalization on ARC-AGI-1. arXiv:2607.06764.

Pourcel, J., Colas, C., and Oudeyer, P.-Y. (2025). Self-improving language models for evolutionary program synthesis: A case study on ARC-AGI. ICML. arXiv:2507.14172.

Snell, C., Lee, J., Xu, K., and Kumar, A. (2024). Scaling LLM test-time compute optimally can be more effective than scaling model parameters. arXiv:2408.03314.

Sorokin, I., and Puget, J.-F. (2025). NVARC solution to ARC-AGI-2 2025. ARC Prize 2025 solution write-up.

Stroebl, B., Kapoor, S., and Narayanan, A. (2024). Inference scaling flaws: The limits of LLM resampling with imperfect verifiers. arXiv:2411.17501.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. ICLR.

Competition reports (Kaggle, ARC-AGI-2; read Nov 5, 2026; author-reported, not reproduced; public-leaderboard ranks).

[K1] markrowset (2026). Out of Reach. Datasets `kaggle.com/datasets/markrowset/arc-agi-ttt-candidate-pools` and `kaggle.com/datasets/markrowset/arc-ttt-coverage-code`, with the notebook `kaggle.com/code/farmountain/arc2-coverage-not-selection` (Liew, K. H.).

[K2] Chen, Y.-C. (2026). Rank 2 post, `kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/755801`.

[K3] Rank 4 (2026). Solution Writeup, `kaggle.com/competitions/arc-prize-2026-arc-agi-2/writeups/Domdolus-Tolus-Solution`, and notebook `kaggle.com/code/darrenamadeusmartin/domdolus-tolus-arc-agi-2`.

[K4] Rank 9 (2026). Solution Writeup, `kaggle.com/competitions/arc-prize-2026-arc-agi-2/writeups/9th-public-lb-place-writeup`.

[K5] Entry scoring 61.25 (2026). Post, `kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/755872`.

[K6] Hiếu Vy (2026). Post, `kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/755245`.

Appendix A. Every intervention, by the side it acts on

Right outputs of 145 on the bank (B0 40, merge 45) unless the data column names another set or the result gives its base ("a to b", same-run bases); "decision" is the outcome under the decision rule fixed before the run.

Selection.

Track	Intervention	Data	Result	Decision
ARC-2	zbeam attempt-2 picker	bank	41; merge 47	rule not met
ARC-2	amax attempt-2 picker	bank	40; merge 45	own rule met
ARC-2	borda attempt-2 picker	390 dev outputs	87 against 86	not chosen
ARC-2	16 or 32 rescoring augmentations	bank, one run	42, 40 against 42	no winner
ARC-2	30B likelihood re-ranking	bank	35 (+1 −6)	declined
ARC-2	2B zero-shot judge	bank	34, 25, 36, 35	dropped
ARC-2	27B checks B0's top two	25 outputs	10 against 14	not adopted
ARC-2	merges conditioned on B0's score margin	90 outputs	net +7 to +9, not +12	merge kept
ARC-2	27B first, or if samples agree	bank	45, 45	no change
ARC-2	verified programs first	25 outputs	15 against 14	recorded
ARC-2	written grids over unverified programs	25 outputs	16 against 13	built

Generation.

Track	Intervention	Data	Result	Decision
ARC-2	B0 floor 0.05	65 outputs	pool 15 to 19, 12 to 14, ×1.66	does not fit
ARC-2	B0 superset search	bank, one run	pool 48 to 52, 42 to 39, ×1.76	no winner
ARC-2	temperature samples	bank, one run	pool 45 to 51, 39 to 39, ×1.99	no gain

Track	Intervention	Data	Result	Decision
ARC-2	128 training sequences	bank; leaderboard	36, ×1.17; leaderboard 31.81	no gain
ARC-2	colour 0 fixed	bank	39; merge 44	no gain
ARC-2	B0 model adapted on 80 tasks	65 outputs	15 against 15	not adopted
ARC-2	27B at a 98,304-token cap	26 outputs	8 against 4	rule met; not submitted
ARC-2	two more 27B samples per task	bank	pool 55 to 61; merge 43	ceiling only
ARC-2	worked programs in the rescue	196 paired draws	net −3	no help
ARC-2	Glimmer 30B in the 27B's place	25 outputs	8; variants 5 to 11 (27B 9 to 14)	stopped
ARC-2	Flash-Next in the 27B's place	25 outputs	4, at 99 tokens/s	dropped
ARC-2	27B as a program-writing agent	65 outputs	3, all also B0's	not adopted

Appendix B. Two earlier solvers

Recurrent transducers. We trained small transducers from scratch on ARC grids, each with per-task test-time training: a GRU, Mamba-2 and a Mamba-2/attention hybrid against a Transformer, at about 0.7M parameters on CPU (7,200 CPU s per model, one seed) and about 21M on GPU (one or two seeds per model, 36M to 285M training tokens). Held-out tasks come from the public training set, with test-time training on grids up to 15×15. The models differed on held-out tasks inherited from ARC-AGI-1, but on the 53 held-out test outputs from tasks new in ARC-AGI-2 every model solved none except one GPU hybrid seed, which solved 1; the two GPU hybrids solved 0 of 62 on the dev split. Limits: 0.7M to 21M parameters, short training, one or two seeds, grids up to 15×15 for test-time training, public tasks.

Program search with learned priors. Our earlier program-search solver searches a typed grammar of scene operations, accepts a program only if it reproduces every demonstration, and ranks the programs that fit by description length. It tried eight small learned priors that predict the kind of program a task needs and reorder the search: logistic regression, a linear SVM, an MLP, an RNN, a GRU, an LSTM, a small state-space cell and a JEPA-inspired model (2,205 to 6,705 parameters). Trained on generated tasks, they named the program kind of held-out generated tasks with 71% to 78% accuracy; none solved any of 85 held-out outputs from real training tasks new in ARC-AGI-2. At 10 s per task the logistic, state-space and JEPA-inspired priors scored 0 of 62 on dev and 0 of 53 on tune, and the logistic prior's answers matched the unguided search's on all 233 new training tasks. In the solver's own oracle diagnostic on one training task, the correct family and perception yield 1,191 programs and no projection proposal. A proposer written after studying that task supplied the program and solved it: a change to generation, and an in-sample one. Limits: the priors read 244 pooled statistics of the demonstrations, not grids; the state-space cell is an input-selective diagonal cell, not Mamba, and the JEPA-inspired model adds a masked latent-prediction loss to an MLP, not a JEPA over grids. Nothing here bears on state-space or JEPA world models at scale.

Track	Solver	Change	Data	Result
ARC-2	recurrent, CPU, 0.7M	GRU, Mamba-2, hybrid vs Transformer	237 held-out outputs ≤ 15×15, test-time training	5, 7, 13 against 15
ARC-2	recurrent, CPU and GPU	every model	53 held-out outputs, tasks new in ARC-AGI-2	0; one GPU hybrid seed 1
ARC-2	recurrent, GPU, 21M	hybrid, two seeds	dev, 62 outputs	0 and 0
ARC-2	recurrent, GPU, 21M	hybrid vs Transformer, equal tokens	237 held-out outputs, as above	38 and 47 against 24
ARC-2	recurrent, GPU, 21M	the same at equal GPU time	237 held-out outputs, as above	38 and 47 against 55 (p 0.008, 0.18)
ARC-2	program search	eight learned priors	85 validation outputs, tasks new in ARC-AGI-2	0 for each
ARC-2	program search	logistic, state-space, JEPA-inspired priors	dev (62) and tune (53), 10 s per task	0 and 0 for each

Track	Solver	Change	Data	Result
ARC-2	program search	oracle family and perception	one training task	1,191 programs, no projection proposal

Table B1: The two earlier solvers. "Dev" and "tune" are 40-task splits of the public evaluation set.

Appendix C. Artifacts and reproducibility

The repository (github.com/maestrosatyam/arc-prize-2026) holds the bank's candidate pools and 27B samples; the loader, rules and scorer (`eval/arc2_bank.py`); the replay, null, bootstrap, profile, calibration, account and figure scripts (`scripts/arc2_*.py`); every rule as fixed and every result file used here (`experiments/arc2-bank/`, `experiments/arc2-pool-audit/`); and a claims table mapping each number to its source. The entry notebooks are public on Kaggle: v7-vr (`purushottamanurag/arc-agi-2-hybrid-v7vr`) and zbeam (`purushottamanurag/arc-agi-2-hybrid-v7vr2-zbeams`), with inputs pinned by version. B0 is the public notebook `mikelou1/arc-agi2-lb33-89-minimal-perfpatch` with the model `sorokin/qwen3_4b_grids15_sft139`. Licence: code under MIT-0; the bank's candidate pools, 27B samples, rules and result files under CC BY 4.0, with attribution to ARC Prize for the ARC-AGI-2 tasks. No model weights are redistributed.